Développer de manière sécurisée et agile — Support de cours (3-033)

Tahar Madi · Ascent Formation

Présentation

Ce support accompagne les deux jours de formation : il reprend les apports de chaque module, avec des exemples tirés de l'application de TP. Gardez-le comme référence pour vos projets.

À l'issue de la formation, vous saurez :

- structurer un projet agile et y intégrer des exigences de sécurité vérifiables ;
- identifier les risques d'une application et les traduire en user stories et critères d'acceptation ;
- insérer des contrôles de sécurité automatisés dans un pipeline CI/CD ;
- analyser leurs résultats, conduire un audit agile et prioriser les corrections.

Jour	Module	Question centrale
J1 matin	1. Introduction au développement agile	Où la sécurité entre-t-elle dans un sprint ?
J1 après-midi	2. La sécurité dans les projets agiles	Quels risques, et comment les écrire en exigences testables ?
J2 matin	3. Pratiques de sécurité dans les outils agiles	Comment le pipeline vérifie-t-il ces exigences ?
J2 après-midi	4. Suivi, audit et gestion des risques	Comment savoir si l'on est en sécurité, et le rester ?

L'application fil rouge. « Demandes internes » est une petite application web Flask : un agent crée, consulte et recherche ses demandes ; un gestionnaire les clôture. Elle contient volontairement des failles courantes. Vous la ferez évoluer en équipe sur GitLab pendant les deux jours, de la découverte des failles à leur correction prouvée par le pipeline.

> *Cette application est vulnérable par construction. Ne la déployez jamais hors de l'environnement de formation.*

Module 1 — Agilité et points d'entrée de la sécurité

En agile, la sécurité ne peut pas être une étape finale : elle doit avoir sa place dans chaque artefact et chaque événement du sprint. Ce module rappelle Scrum et Kanban, puis montre où la loger.

1.1 Cycle en V et cycle itératif

	Cycle en V	Cycle itératif (agile)
Principe	Spécifier, concevoir, réaliser,	Livrer de petits incréments vérifiés,

	Cycle en V	Cycle itératif (agile)
	puis vérifier	ajuster à chaque itération
Moment de la sécurité	Exigences au début, audit ou test d'intrusion à la fin	À chaque incrément, si on l'organise
Risque principal	Défaut de conception découvert tard, donc coûteux	« On traitera la sécurité au prochain sprint » : la dette s'accumule
Atout	Exigences formalisées d'emblée	Retour rapide, correction peu coûteuse

L'agilité n'est donc ni plus ni moins sûre en soi. Elle le devient quand la sécurité entre dans ses règles : backlog, critères d'acceptation, Definition of Done, pipeline.

1.2 Scrum en bref

Scrum est un cadre léger, décrit par le Scrum Guide (version 2020), qui organise le travail en sprints d'un mois au plus.

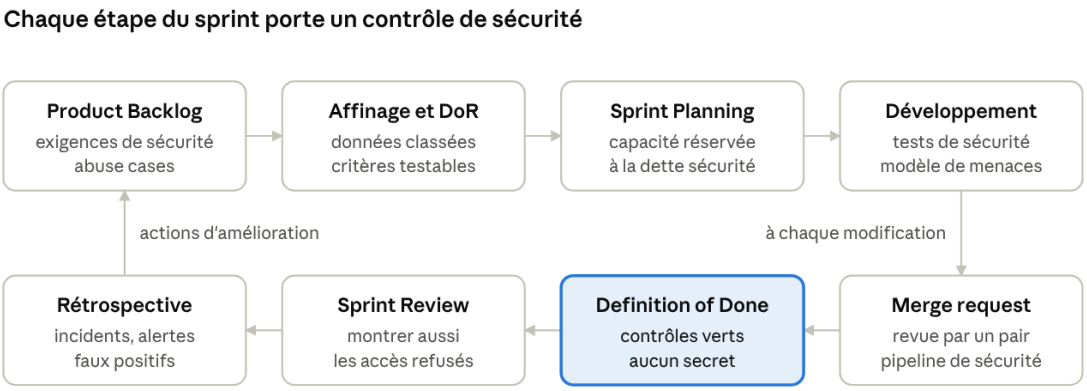
- **Trois responsabilités.** Le *Product Owner* maximise la valeur du produit et ordonne le Product Backlog. Le *Scrum Master* fait vivre le cadre et retire les obstacles. Les *Developers* construisent l'incrément et s'engagent sur sa qualité.
- **Cinq événements.** Le Sprint lui-même ; le Sprint Planning (quoi et comment) ; le Daily Scrum (15 minutes, synchronisation) ; la Sprint Review (l'incrément montré aux parties prenantes) ; la Sprint Retrospective (améliorer le fonctionnement).
- **Trois artefacts et leur engagement.** Product Backlog et *Product Goal* ; Sprint Backlog et *Sprint Goal* ; Increment et *Definition of Done*.

1.3 Kanban en bref

Kanban gère un flux continu plutôt que des itérations. Ses pratiques : visualiser le travail sur un tableau, limiter le travail en cours (WIP), gérer le flux, rendre les règles explicites, mesurer (délai de bout en bout, temps de cycle, débit). Un couloir d'urgence (*expedite*) permet de traiter une vulnérabilité critique sans casser le flux.

Scrum et Kanban se combinent souvent : sprints Scrum, tableau Kanban avec limites de WIP.

1.4 La sécurité à chaque étape du sprint



le sprint Scrum et ses contrôles de sécurité · 8 étapes

La Definition of Done est le point d'engagement : une story qui ne passe pas ses contrôles de sécurité n'est pas terminée. Les actions de la rétrospective repartent dans le backlog.

Le champion sécurité est un développeur de l'équipe, formé et en lien avec le RSSI. Il anime la modélisation des menaces, aide au tri des alertes et diffuse les bonnes pratiques. Il ne porte pas seul la sécurité : elle reste l'affaire de toute l'équipe.

1.5 User stories, *abuse cases* et critères d'acceptation

Une user story décrit un besoin du point de vue d'un utilisateur : « En tant que …, je veux …, afin de … ». Une bonne story respecte les critères INVEST : indépendante, négociable, porteuse de valeur, estimable, petite (*small*), testable.

L'***abuse case*** retourne la story du point de vue de l'attaquant :

> *En tant qu'agent malveillant, je veux ouvrir les demandes de mes collègues en changeant le numéro dans l'URL, afin de lire leurs données.*

Il produit des **critères d'acceptation négatifs**, c'est-à-dire ce que le système doit refuser. Le format Étant donné / Quand / Alors (Gherkin) les rend directement testables :

```
Scénario : un agent ne lit pas la demande d'un autre agent
  Étant donné que bob est connecté
  Quand il ouvre la demande n° 1, qui appartient à alice
  Alors il reçoit une erreur 404
```

1.6 Definition of Ready et Definition of Done

Definition of Ready : une story entre en sprint si…	Definition of Done : une story est terminée si…
les données manipulées sont identifiées et classées	les tests, dont les tests de sécurité, sont verts dans le pipeline
les rôles autorisés sont précisés	les contrôles SAST, dépendances et secrets ne signalent aucun défaut bloquant
les critères de sécurité sont écrits et testables	la merge request est approuvée par une autre personne
le besoin de modélisation des menaces est tranché	aucun secret ni donnée réelle dans le code, les tests et les journaux
la story est estimée	le modèle de menaces est à jour si un flux a changé

Un critère comme « l'application est sécurisée » n'a pas sa place dans une DoD : il n'est pas vérifiable.

À retenir

- La sécurité vit dans les artefacts agiles : backlog, critères d'acceptation, DoR, DoD.
- Un *abuse case* transforme une menace en critère d'acceptation négatif et testable.
- Une story qui ne passe pas ses contrôles de sécurité n'est pas terminée.

Module 2 — Principes de sécurité et DevSecOps

La plupart des failles graves viennent de quelques erreurs récurrentes : contrôle d'accès absent, données non validées, cryptographie mal choisie, secrets exposés. Ce module les illustre sur l'application fil rouge, puis montre comment les anticiper par la modélisation des menaces et les traduire en stories.

2.1 Gestion des accès

Trois mécanismes distincts, souvent confondus :

- **L'authentification** répond à « qui êtes-vous ? » : mot de passe, multifacteur (MFA) pour les comptes à privilèges.
- **La session** maintient cette identité entre les requêtes : cookie `HttpOnly` (illisible par JavaScript), `Secure` (HTTPS seulement), `SameSite` (limite les requêtes intersites), avec une expiration.
- **L'autorisation** répond à « avez-vous le droit ? ». Elle se vérifie côté serveur, à chaque requête, sur l'objet demandé, avec un refus par défaut et le moindre privilège.

L'erreur la plus fréquente est d'authentifier sans autoriser. Dans l'application de TP, tout utilisateur connecté peut lire n'importe quelle demande :

```
# Vulnérable : seule la connexion est vérifiée
demande = db.execute("SELECT * FROM demandes WHERE id = ?",
```

```
(demande_id,)).fetchone()
if demande is None:
    abort(404)
```

```
# Corrigé : on vérifie que la demande appartient à l'utilisateur (ou qu'il
est gestionnaire)
if demande is None or (
    demande["owner_id"] != user["id"] and user["role"] != "gestionnaire"
):
    abort(404)  # 404 plutôt que 403 : on ne révèle pas que la demande existe
```

Ce défaut porte un nom : IDOR (*Insecure Direct Object Reference*). Aucun scanner automatique ne le détecte de façon fiable, car il dépend d'une règle métier. Seuls la conception et les tests le préviennent.

2.2 Cryptographie

- **Hacher n'est pas chiffrer.** Un hachage est à sens unique : il sert à vérifier un mot de passe. Un chiffrement est réversible avec une clé : il sert à protéger une donnée qu'on doit relire, comme un RIB.
- **Mots de passe** : algorithme lent et salé, conçu pour cet usage (argon2id, scrypt, bcrypt, PBKDF2). MD5 et SHA-1 se calculent trop vite : une empreinte de mot de passe courant se retrouve en quelques secondes dans un dictionnaire.
- **En transit** : TLS 1.2 au minimum, 1.3 de préférence, y compris entre services internes.
- **Aléa** : module `secrets` de Python, jamais `random` pour un jeton ou une clé.
- **Ne pas inventer** : bibliothèques standard et paramètres par défaut récents.

```
# Vulnérable
def hash_password(password):
    return hashlib.md5(password.encode()).hexdigest()

# Corrigé (Werkzeug utilise scrypt par défaut)
from werkzeug.security import generate_password_hash, check_password_hash
def hash_password(password):
    return generate_password_hash(password)
```

2.3 Injection et XSS

Une **injection** survient quand une donnée fournie par l'utilisateur est interprétée comme du code : SQL, commande système, HTML. La règle : ne jamais concaténer une saisie dans une requête, toujours la passer en paramètre.

```
# Vulnérable : la saisie q est insérée dans le texte SQL
sql = "SELECT * FROM demandes WHERE owner_id = %d AND titre LIKE '%%%s%%'" %
(user["id"], q)
demandes = db.execute(sql).fetchall()

# Corrigé : requête paramétrée
demandes = db.execute(
    "SELECT * FROM demandes WHERE owner_id = ? AND titre LIKE ?",
    (user["id"], f"%{q}%"),
).fetchall()
```

Avec la version vulnérable, la recherche `' UNION SELECT id, 0, username || ':' || password, '', role FROM users --` affiche tous les comptes et leurs empreintes de mot de passe.

Le **cross-site scripting (XSS)** est une injection dans la page HTML. Les moteurs de gabarits comme Jinja échappent automatiquement les données ; le filtre `| safe` désactive cette protection et ne doit jamais s'appliquer à une saisie utilisateur.

2.4 OWASP Top 10:2025

L'OWASP Top 10 classe les risques les plus fréquents des applications web. L'édition 2025 introduit deux catégories : la chaîne d'approvisionnement logicielle (A03) et la mauvaise gestion des situations exceptionnelles (A10).

Rang	Catégorie	Exemple dans l'application de TP
A01	Broken Access Control	Lire ou clôturer la demande d'un autre agent
A02	Security Misconfiguration	Mode debug actif, en-têtes de sécurité absents
A03	Software Supply Chain Failures	Flask, Werkzeug et Jinja2 en versions vulnérables
A04	Cryptographic Failures	Mots de passe hachés en MD5
A05	Injection	Injection SQL dans la recherche, XSS dans la description
A06	Insecure Design	Aucune règle d'accès prévue dès la conception
A07	Authentication Failures	Essais de mot de passe illimités
A08	Software or Data Integrity Failures	Image Docker non épinglée
A09	Security Logging and Alerting Failures	Mot de passe écrit dans les journaux
A10	Mishandling of Exceptional Conditions	Erreurs détaillées exposées en mode debug

Pour écrire des exigences précises, l'**OWASP ASVS 5.0** (*Application Security Verification Standard*) propose des centaines d'exigences vérifiables en trois niveaux. Une application interne manipulant des données personnelles vise en général le niveau 2.

2.5 Gérer les vulnérabilités

La gestion des vulnérabilités est un cycle, pas une campagne :

1. **Identifier** : scanners, revues, veille (avis du CERT-FR, CVE).
2. **Évaluer** : gravité (CVSS), exploitation avérée, exposition de l'application.
3. **Traiter** : corriger, atténuer, ou accepter de façon signée et datée.
4. **Vérifier** : la correction est prouvée par un test ou un nouveau scan.
5. **Suivre** : indicateurs et délais (module 4).

2.6 DevSecOps et *shift left*

DevSecOps intègre la sécurité à la chaîne de développement : automatisée, continue et partagée par l'équipe, plutôt qu'un contrôle final par une équipe séparée. *Shift left* signifie déplacer les contrôles vers le début du cycle : un défaut trouvé dans l'éditeur ou la merge request se corrige en minutes ; le même défaut trouvé en production coûte un incident.

Étape	Contrôle	Exemple d'outil
Poste du développeur	Analyse dans l'éditeur, crochet anti-secrets avant commit	SonarQube for IDE, Gitleaks
Merge request	Analyse du code, des dépendances, des secrets ; revue	Semgrep, Bandit, pip-audit, Gitleaks
Build	Inventaire des composants, analyse d'image	CycloneDX, Trivy
Environnement de test	Analyse dynamique	OWASP ZAP
Production	Journaux, alertes, veille	SIEM, CERT-FR

2.7 Modéliser les menaces

La modélisation des menaces consiste à se demander, avant de coder, ce qui peut mal tourner. Adam Shostack la résume en quatre questions :

1. Sur quoi travaillons-nous ? — un schéma des flux : acteurs, processus, stockages, flux, frontières de confiance.
2. Qu'est-ce qui peut mal tourner ? — STRIDE appliqué à chaque élément.
3. Que faisons-nous face à cela ? — des mesures, donc des stories.
4. Avons-nous bien travaillé ? — relecture et critère de sortie.

Lettre	Menace	Propriété violée	Question à poser
S	Usurpation d'identité (*Spoofing*)	Authentification	Quelqu'un peut-il se faire passer pour un autre ?
T	Altération (*Tampering*)	Intégrité	Peut-on modifier une donnée ou une requête ?
R	Répudiation	Traçabilité	Peut-on nier avoir agi, faute de trace ?
I	Divulgation d'information	Confidentialité	Une donnée peut-elle fuiter ?
D	Déni de service	Disponibilité	Peut-on rendre le service indisponible ?
E	Élévation de privilèges	Autorisation	Peut-on faire ce que son rôle n'autorise pas ?

En agile, on la pratique légère et souvent : à chaque story qui touche une frontière de confiance (nouvelle saisie, nouveau flux, nouvelle donnée sensible), en 15 à 30 minutes lors de l'affinage. L'analyse de risque du système entier relève d'une méthode plus complète comme EBIOS Risk Manager (ANSSI).

2.8 Des menaces aux *security user stories*, puis aux tests

Chaque menace retenue devient une story de sécurité avec au moins un scénario d'abus. Ce scénario se traduit ensuite, presque mot pour mot, en test automatisé :

> ***SEC-01** — En tant qu'agent, je veux que mes demandes ne soient visibles que par moi et les gestionnaires, afin de protéger leur contenu. Étant donné bob connecté, quand il ouvre la demande n° 1 d'alice, alors il reçoit une erreur 404.*

```
def test_un_agent_ne_voit_pas_la_demande_d_un_autre_agent(client, login):
    login("bob", "bob2026")                                # Étant donné bob
connecté
    reponse = client.get("/demandes/1")                    # Quand il ouvre la
demande d'alice
    assert reponse.status_code == 404                      # Alors il reçoit
une 404
```

Ce test échoue tant que la faille existe et passe une fois corrigée. Exécuté dans le pipeline, il empêche la faille de revenir : c'est le lien entre la conception agile et la chaîne DevSecOps.

À retenir

- Authentifier n'est pas autoriser : l'autorisation se vérifie sur chaque objet, à chaque requête.
- Toute saisie est une donnée, jamais du code : requêtes paramétrées, échappement HTML.
- Les failles de conception échappent aux scanners : la modélisation des menaces et les tests issus des stories les attrapent.

Module 3 — Sécuriser la chaîne et automatiser les contrôles

La chaîne de développement est elle-même une cible : un dépôt mal protégé, un secret dans le code ou une dépendance piégée suffisent à compromettre une application sans toucher à son code. Ce module montre comment sécuriser cette chaîne, puis lui faire vérifier automatiquement chaque modification.

3.1 Environnements et configurations

- **Séparer** développement, test et production ; ne jamais copier de données réelles en test.
- **Externaliser la configuration** dans des variables d'environnement, pas dans le code. Le même artefact est déployé partout, seule la configuration change.
- **Désactiver le mode debug** hors du poste du développeur : le débogueur de Flask permet d'exécuter du code à distance.
- **Durcir les images** : image minimale (`python:3.12-slim`), utilisateur non root, versions épinglées.
- **Versionner l'infrastructure** (*infrastructure as code*) et la relire comme le reste du code.

3.2 Les secrets

Un secret (mot de passe, jeton d'API, clé privée) poussé dans Git est compromis, même si on le supprime au commit suivant : l'historique le conserve, et chaque clone en garde une copie.

Étape	Bonne pratique
Stocker	Variables CI/CD *masquées* (absentes des journaux) et *protégées* (réservées aux branches protégées), ou coffre (HashiCorp Vault, OpenBao, coffre du cloud)
Prévenir	Gitleaks en crochet de pré-commit sur le poste
Détecter	Gitleaks dans le pipeline, sur tout l'historique
Réagir	**Révoquer** le secret et en émettre un nouveau ; réécrire l'historique ne suffit pas

3.3 Droits et branches dans GitLab

- **Rôles au plus juste** : Reporter (lecture), Developer (contribution), Maintainer (administration du projet). MFA obligatoire ; jetons d'accès à durée limitée.
- **Branche principale protégée** : aucun push direct, fusion uniquement par merge request.
- **« Le pipeline doit réussir »** avant toute fusion : c'est ce qui rend les contrôles bloquants.
- **Revue par un pair** sur chaque merge request.
- **Runners** : jobs non privilégiés, runners dédiés et protégés pour les déploiements, secrets visibles seulement des jobs qui en ont besoin.

3.4 La chaîne d'approvisionnement logicielle

Une application moderne est surtout faite de code qu'on n'a pas écrit : bibliothèques, images, actions de CI. Chacun de ces composants peut être compromis.

Date	Incident	Leçon
Mars 2026	Trivy, scanner de vulnérabilités : une version du binaire piégée et 75 des 76 étiquettes de son action GitHub déplacées vers un code qui volait les secrets des pipelines	Un outil de sécurité est aussi une dépendance ; une étiquette peut être déplacée, une empreinte non
Mars 2025	Action GitHub `tj-actions/changed-files` compromise : secrets de CI exposés dans les journaux	Épingler par empreinte ; limiter les secrets visibles par chaque job
Mars 2024	Porte dérobée dans XZ Utils (CVE-2024-3094), introduite après des mois de contributions légitimes	Savoir répondre vite à « sommes-nous touchés ? »

Les parades :

- **Épingler** chaque dépendance et chaque image sur une version, et en production sur une empreinte (`image@sha256:…`).

- **Inventorier** les composants dans un SBOM (*Software Bill of Materials*) au format CycloneDX ou SPDX.
- **Filtrer** les dépendances entrantes par un dépôt proxy (Nexus Repository, Artifactory).
- **Signer** les artefacts produits (Sigstore).

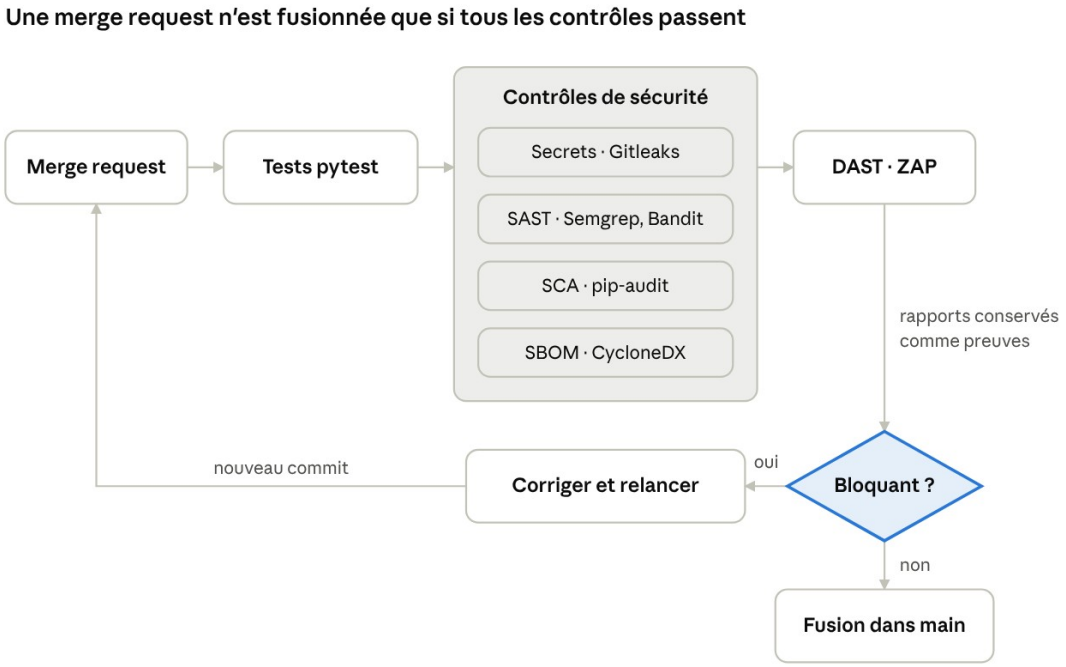
Le *Cyber Resilience Act* européen impose aux fabricants de produits comportant des éléments numériques un SBOM et, depuis le 11 septembre 2026, le signalement sous 24 heures des vulnérabilités activement exploitées.

3.5 Les familles de tests de sécurité

Famille	Ce qu'elle analyse	Forces	Angles morts	Outils
SAST	Le code source, sans l'exécuter	Tôt, ligne précise	Faux positifs ; ignore la logique métier	Semgrep, Bandit, SonarQube
SCA	Les dépendances, comparées aux bases de vulnérabilités	Rapide, fiable	Ne dit pas si le code vulnérable est appelé	pip-audit, OWASP Dependency-Check, Trivy
Secrets	Le code et l'historique Git	Très fiable sur les formats connus	Secrets au format inconnu	Gitleaks, TruffleHog
SBOM	L'inventaire des composants	Réponse rapide lors d'une alerte	N'est pas un contrôle en soi	CycloneDX, Syft
DAST	L'application en fonctionnement	Voit la configuration réelle	Arrive tard ; couverture limitée à ce qu'il explore	OWASP ZAP, Burp Suite
Conteneurs et IaC	Images, Dockerfile, Terraform	Configuration d'exécution	—	Trivy, Checkov, Hadolint

Sur l'application fil rouge, ces outils trouvent l'injection SQL, le hachage MD5, le mode debug, le secret et les dépendances vulnérables. **Aucun ne trouve l'accès aux demandes d'un autre agent** : c'est le rôle des tests issus des stories.

3.6 Le pipeline de référence



pipeline de référence du projet fil rouge · 3 étapes, 1 seuil

Les rapports de chaque job sont conservés comme artefacts : ce sont les preuves exploitées lors de l'audit (module 4). Extrait du job d'analyse des dépendances :

```
sca-pip-audit:
  stage: securite
  image: python:3.12-slim          # image épinglée
  script:
    - pip install -q pip-audit==2.10.1
    - mkdir -p rapports
    - pip-audit -r requirements.txt -f json -o rapports/pip-audit.json ||
true   # rapport complet
    - pip-audit -r requirements.txt --desc
# échoue si vulnérabilité
  allow_failure: false             # seuil bloquant
  artifacts:
    when: always
    paths: [rapports/pip-audit.json]
```

3.7 Seuils et faux positifs

- **Commencer en observation** (`allow_failure: true`) pour mesurer le bruit, puis **bloquer** sur les défauts critiques et élevés.
- **Bloquer d'abord le nouveau** : un défaut introduit par la merge request doit être corrigé ; l'existant passe par le backlog.

- **Tracer chaque exception** : un faux positif ignoré l'est avec une justification (`# nosec B104 — écoute requise dans le conteneur`, empreinte dans `.gitleaksignore` avec date et ticket).
- **Éviter la fatigue d'alerte** : un pipeline toujours rouge finit contourné.

3.8 Gérer les vulnérabilités dans un cadre agile

1. **Trier à l'arrivée** : vrai ou faux positif ? atteignable ? exploité dans la nature ?
2. **Créer un ticket** `type::securite` avec criticité et preuve (lien vers le rapport du job).
3. **Appliquer un délai par criticité**, fixé par la politique de l'organisation (par exemple : critique 7 jours, élevée 30 jours, moyenne 90 jours).
4. **Réserver de la capacité** dans chaque sprint ; une vulnérabilité critique exploitée passe en couloir d'urgence.
5. **Fermer avec une preuve** : pipeline vert et test de non-régression.

À retenir

- Un secret poussé est compromis : on le révoque, on ne se contente pas de l'effacer.
- Épingler, inventorier, filtrer : la chaîne d'approvisionnement se sécurise comme le code.
- Chaque famille d'outils a ses angles morts ; le pipeline les combine et bloque sur un seuil assumé.

Module 4 — Suivi, audit et gestion des risques

Une application livrée sans défaut connu ne le reste pas : de nouvelles vulnérabilités sont publiées chaque semaine sur des composants déjà en production. Ce module montre comment surveiller après la livraison, piloter les risques et vérifier, par un audit léger, que les pratiques tiennent dans la durée.

4.1 La surveillance continue

Quoi	Comment	Outil
Nouvelles vulnérabilités des dépendances	Pipeline planifié chaque nuit, même sans commit	GitLab, *pipeline schedules*
Composants livrés	Le SBOM de chaque version est suivi et comparé aux nouvelles CVE	OWASP Dependency-Track
Vulnérabilités de toutes sources	Centralisation, dédoublonnage, suivi des délais	DefectDojo, rapport de vulnérabilités GitLab
Comportement de l'application	Journaux de sécurité : connexions échouées, accès refusés, actions des gestionnaires, avec alertes	SIEM
Menaces nouvelles	Veille sur les composants utilisés	Avis et alertes du

Quoi	Comment	Outil
		CERT-FR

Un bon journal de sécurité dit **qui** a fait **quoi**, **quand** et avec **quel résultat**, sans jamais contenir de mot de passe, de jeton ou de donnée sensible.

4.2 Prioriser les vulnérabilités

Le score CVSS seul ne suffit pas : beaucoup de vulnérabilités « critiques » ne sont jamais exploitées, et certaines « moyennes » le sont massivement. On croise trois signaux avec le contexte :

Signal	Ce qu'il mesure	Source
CVSS v4.0	La gravité technique, dans l'absolu	NVD, éditeur
EPSS	La probabilité d'exploitation dans les 30 jours	FIRST
KEV	Le fait que la vulnérabilité soit déjà exploitée	Catalogue de la CISA
Contexte	Exposition sur Internet, données touchées, mesures compensatoires	L'équipe

Règle pratique : une vulnérabilité présente dans le catalogue KEV, ou à EPSS élevé, sur un composant exposé, passe en couloir d'urgence quel que soit son score CVSS.

4.3 Les indicateurs

Des indicateurs simples, suivis en Sprint Review ou en comité sécurité, valent mieux qu'un tableau de bord exhaustif. On regarde la tendance plutôt qu'une cible absolue.

- Vulnérabilités ouvertes par criticité et par ancienneté.
- Délai moyen de correction, comparé au délai fixé pour chaque criticité.
- Part des stories comportant des critères de sécurité.
- Part des merge requests passées par le pipeline complet.
- Secrets détectés, taux de faux positifs, nombre de tests de sécurité.

4.4 Le registre de risques

Un risque se formule en chaîne : **cause → événement → conséquence**. Par exemple : « l'absence de jeton anti-CSRF (cause) permet à un site tiers de faire clôturer une demande par un gestionnaire connecté (événement), ce qui fausse le suivi des demandes (conséquence) ».

On le cote par sa vraisemblance et son impact, de 1 à 4. Le score est leur produit :

Score	Niveau	Délai indicatif
12 à 16	Critique	Avant toute mise en production
8 à 11	Élevé	Sprint en cours

Score	Niveau	Délai indicatif
4 à 7	Moyen	Version en cours
1 à 3	Faible	Backlog, ou acceptation signée

Quatre traitements possibles : **réduire** (une mesure, donc une story), **transférer** (assurance, prestataire), **éviter** (renoncer à la fonctionnalité), **accepter** (décision signée par le Product Owner, avec une date de réexamen). En agile, le registre vit à côté du backlog : chaque risque à réduire y a son ticket, et le registre est revu à chaque version.

4.5 Les données sensibles sur tout le cycle de vie

Commencez par **identifier et classer** les données : données personnelles (RGPD), catégories particulières (santé, opinions…), données financières, secrets techniques, informations métier confidentielles. Puis appliquez une mesure à chaque étape :

Étape	Pratique
Conception	Minimiser ce qui est collecté ; protection des données dès la conception (RGPD, art. 25) ; analyse d'impact si le risque est élevé (art. 35)
Développement et tests	Données fictives ou anonymisées, jamais de copie de production ; aucune donnée réelle dans les tickets ou les captures d'écran
Stockage	Chiffrement des champs sensibles et des sauvegardes ; accès au plus juste
Transit	TLS partout, y compris entre services internes
Journalisation	Ni mot de passe, ni jeton, ni RIB ; identifiants pseudonymisés si possible
Conservation	Durée définie par type de donnée ; purge automatisée
Fin de vie	Suppression vérifiable, y compris dans les sauvegardes et les exports

En agile, ces règles s'ancrent dans un label `donnees::sensibles` sur les stories, dans la Definition of Ready (« données identifiées et classées ») et dans la Definition of Done (« aucune donnée réelle dans les tests et les journaux »). Dans le secteur public, elles alimentent aussi le dossier d'homologation de sécurité.

4.6 L'audit agile de la sécurité

L'audit agile n'est pas un contrôle lourd de fin de projet, mais une vérification régulière, à chaque version ou chaque trimestre.

- **Il s'appuie sur des preuves déjà dans les outils** : tickets, merge requests, pipelines, rapports. Ce qui n'est pas prouvé est noté absent.
- **Il gradue la maturité** sur une échelle simple : 0 absent, 1 initié, 2 en place, 3 en place et prouvé. Pour aller plus loin : OWASP SAMM, OWASP DSOMM.
- **Il classe ses constats** : critique (exploitable, impact fort, à traiter avant mise en production), élevé (dans le sprint), moyen (dans la version), faible (backlog).
- **Il produit des recommandations actionnables** : chacune devient une story ou une règle de la Definition of Done, avec un responsable et une échéance.

Exemples de questions d'audit : la branche principale est-elle protégée ? Chaque story de sécurité a-t-elle un critère testable, couvert par un test ? Les contrôles du pipeline sont-ils bloquants ? Les vulnérabilités sont-elles triées avec un délai ? Les données sensibles sont-elles absentes des journaux ?

4.7 L'amélioration continue

La rétrospective est l'endroit où la sécurité progresse : incidents, alertes, faux positifs et constats d'audit y sont analysés, et chaque action d'amélioration retourne dans le backlog. Une équipe qui ne parle jamais de sécurité en rétrospective n'apprend pas de ses incidents.

À retenir

- Une application sûre aujourd'hui ne l'est pas demain : surveillez après la livraison.
- Priorisez par l'exploitation réelle (KEV, EPSS) et le contexte, pas par le seul score CVSS.
- Un audit agile est léger, régulier, fondé sur des preuves, et se termine en stories.

Glossaire

Terme	Définition
Abuse case	User story écrite du point de vue d'un attaquant, qui décrit ce que le système doit empêcher
ASVS	*Application Security Verification Standard* (OWASP) : référentiel d'exigences de sécurité vérifiables, en trois niveaux
Champion sécurité	Développeur de l'équipe formé à la sécurité, relais entre l'équipe et le RSSI
CVE	Identifiant public d'une vulnérabilité connue (ex. CVE-2024-3094)
CVSS	*Common Vulnerability Scoring System* : score de gravité technique d'une vulnérabilité, de 0 à 10
DAST	*Dynamic Application Security Testing* : test d'une application en fonctionnement
DevSecOps	Intégration de la sécurité, automatisée et partagée, dans toute la

Terme	Définition
	chaîne de développement et d'exploitation
DoD / DoR	*Definition of Done* : conditions pour qu'une story soit terminée. *Definition of Ready* : conditions pour qu'elle entre en sprint
EPSS	*Exploit Prediction Scoring System* (FIRST) : probabilité qu'une vulnérabilité soit exploitée dans les 30 jours
IDOR	*Insecure Direct Object Reference* : accès à l'objet d'un autre utilisateur en modifiant son identifiant
KEV	*Known Exploited Vulnerabilities* : catalogue de la CISA des vulnérabilités exploitées
Pipeline CI/CD	Enchaînement automatisé de construction, tests et déploiement, déclenché à chaque modification
Runner	Programme qui exécute les jobs d'un pipeline GitLab
SAST	*Static Application Security Testing* : analyse du code source sans l'exécuter
SBOM	*Software Bill of Materials* : inventaire des composants d'un logiciel (formats CycloneDX, SPDX)
SCA	*Software Composition Analysis* : analyse des dépendances au regard des vulnérabilités connues
Seuil de blocage	Règle qui fait échouer le pipeline au-delà d'un niveau de gravité (*quality gate*)
Shift left	Déplacer les contrôles de sécurité vers le début du cycle de développement
STRIDE	Grille de menaces : usurpation, altération, répudiation, divulgation, déni de service, élévation de privilèges
WIP	*Work In Progress* : travail en cours, que Kanban limite
XSS	*Cross-Site Scripting* : injection de code dans une page web consultée par d'autres utilisateurs

Pour aller plus loin

Thème	Référence
Agilité	Scrum Guide 2020
Risques applicatifs	OWASP Top 10:2025

Thème	Référence
Exigences vérifiables	OWASP ASVS
Maturité et audit	OWASP SAMM, OWASP DSOMM
Modélisation des menaces	Les 4 questions d'Adam Shostack, EBIOS Risk Manager (ANSSI)
Chaîne d'approvisionnement	Wiz — compromission de Trivy, mars 2026
Réglementation	Commission européenne — signalements au titre du Cyber Resilience Act
Veille	CERT-FR
Priorisation	EPSS (FIRST), catalogue KEV (CISA)
Suivi continu	OWASP Dependency-Track, DefectDojo
Outils des TP	Gitleaks, Semgrep, Bandit, pip-audit, CycloneDX, OWASP ZAP, Burp Suite Community